

Применение генетического алгоритма для решения метрической задачи коммивояжёра

Ляшенко Александр Б05-425

3 мая 2026 г.

Аннотация

Данная работа посвящена исследованию генетического алгоритма как метода решения метрической задачи коммивояжёра. В проекте доказывается **NP**-трудность общей задачи коммивояжёра(и показывается, почему не существует приближенного решения) и метрической версии задачи. В практической части реализуется генетический алгоритм, рассматриваются и сравниваются разные виды мутаций(Swap и Inversion) и инициализации(случайная и жадная). Также проводится тестирование алгоритма на различных наборах данных:малые графы, случайные графы, кластерные графы, кольцевые графы. Результаты работы алгоритма сравниваются с детерминированным алгоритмом на основе минимального остовного дерева, который дает 2-приближение и с алгоритмом 2-Opt. В конце показывается, что генетический алгоритм значительно превосходит базовые подходы по качеству решения, хотя и требует больших вычислительных затрат.

1 Введение

Задача коммивояжёра является одной из самых известных и изученных задач оптимизации. Она заключается в поиске кратчайшего маршрута, проходящего через определенный набор городов с возвращением в исходную точку. В данной работе рассматривается метрический вариант задачи, где веса рёбер удовлетворяют неравенству треугольника.

Классические алгоритмы не могут решать эту задачу для графов большого размера за полиномиальное время, но есть приближённые алгоритмы, например, алгоритм на основе остовного дерева, который даёт 2-приближение или алгоритм Кристофидеса который даёт 1.5-приближение. Но на практике эвристические методы часто находят решения, гораздо более оптимальные. В рамках данного проекта реализован и проанализирован генетический алгоритм для решения метрической TSP. Общие принципы построения генетических алгоритмов освещены в литературе [2].

В разделе 2 представлен теоретический анализ сложности задачи, в разделе 3 описаны реализованные алгоритмы, в разделе 4 приведены результаты экспериментов и их анализ, в разделе 5 - заключение. В приложении помещён исходный код и полный вывод программы.

2 Теоретический анализ сложности

2.1 Формальное определение метрической задачи коммивояжёра

Определение 1 Полным взвешенным графом называется тройка $G = (V, E, w)$, где V - множество вершин, E - множество рёбер, $w : E \rightarrow \mathbb{R}^+$ - весовая функция.

Определение 2 Гамильтоновым циклом в графе G называется цикл, проходящий через каждую вершину множества V ровно один раз.

Определение 3 Метрической задачей коммивояжёра называется задача поиска гамильтонова цикла минимального суммарного веса в полном взвешенном графе G , в котором для любых трёх вершин $x, y, z \in V$ выполняется неравенство треугольника:

$$w(x, z) \leq w(x, y) + w(y, z)$$

2.2 Трудность аппроксимации общей задачи коммивояжёра

Определение 4 Алгоритм A для задачи минимизации называется α -приближённым (или имеет коэффициент приближения α), если для лю-

бого входа I он выдаёт решение стоимости $A(I) \leq \alpha \cdot O(I)$, где $O(I)$ - стоимость оптимального решения.

Покажем, почему общую задачу коммивояжёра (без неравенства треугольника) нельзя приближенно решить.

Теорема 1 ([1]) Для стандартной задачи коммивояжёра задача нахождения α -приближённого решения является **NP**-трудной для любого $\alpha \geq 1$.

Доказательство. Докажем **NP**-трудность, построив полиномиальное сведение задачи о гамильтоновом цикле к задаче распознавания α -приближения. Рассмотрим **NP**-полную задачу о гамильтоновом цикле: дан невзвешенный граф $G' = (V', E')$. Сведём её к TSP. Построим полный взвешенный граф $G = (V, E, w)$, где $V = V'$. Веса рёбер определим так:

$$w(u, v) = \begin{cases} 1, & \text{если } (u, v) \in E' \\ \alpha|V| + 1, & \text{если } (u, v) \notin E'. \end{cases}$$

Построение матрицы весов требует $O(|V|^2)$ операций, следовательно, данное преобразование выполняется за полиномиальное время. Покажем эквивалентность решений:

- Если в G' существует гамильтонов цикл, то в графе G оптимальный путь имеет вес $O = |V|$. Тогда любой α -приближённый алгоритм должен найти путь весом не более $\alpha|V|$.
- Если же в G' нет гамильтонова цикла, любой гамильтонов цикл в G будет содержать хотя бы одно ребро веса $\alpha|V| + 1$, и его общий вес будет строго больше $\alpha|V|$.

Получается, построена полиномиальная сводимость **NP**-полного языка. Это означает, что задача приближения общей TSP является **NP**-трудной. ■

Замечание. Из доказанной **NP**-трудности следует, что при условии $P \neq NP$ не существует полиномиального алгоритма для нахождения α -приближённого решения общей задачи коммивояжёра. В противном случае мы получили бы полиномиальный алгоритм для **NP**-полной задачи о гамильтоновом цикле, что означало бы $P = NP$.

Теорема 2 ([1]) *Метрическая задача коммивояжёра (Δ -TSP) является NP-трудной.*

Доказательство. Построим сведение из NP-полной задачи о поиске гамильтонова цикла в невзвешенном графе $G' = (V', E')$. На основе G' построим полный взвешенный граф $G = (V, E, w)$, где $V = V'$. Веса рёбер в G зададим следующим образом:

$$w(u, v) = \begin{cases} 1, & \text{если } (u, v) \in E' \\ 2, & \text{если } (u, v) \notin E' \end{cases}$$

В графе G выполняется неравенство треугольника: для любых трёх вершин x, y, z веса рёбер могут быть только 1 или 2. Возможные комбинации весов сторон треугольника: $(1, 1, 1)$, $(1, 1, 2)$, $(1, 2, 2)$ и $(2, 2, 2)$. Во всех этих случаях сумма любых двух сторон не меньше третьей. Следовательно, полученный граф является метрическим.

Теперь покажем эквивалентность решений:

1. Если в исходном графе G' существует гамильтонов цикл, то в построенном графе G этот цикл состоит из рёбер весом 1. Тогда общая длина такого маршрута равна $|V|$. Это минимально возможный вес, так как все рёбра ≥ 1 .

2. Если гамильтонова цикла в G' нет, то любой гамильтонов цикл в полном графе G должен содержать хотя бы одно ребро, которого не было в E' . Вес такого ребра равен 2. Тогда общая длина пути будет как минимум $(|V| - 1) \cdot 1 + 2 = |V| + 1$.

Таким образом, если бы мы могли решить Δ -TSP за полиномиальное время, мы бы смогли за то же время определить наличие гамильтонова цикла в любом графе G' (проверив, равен ли ответ $|V|$). Значит, Δ -TSP является NP-трудной. ■

3 Техническая часть

В рамках проекта были реализованы два подхода: детерминированный алгоритм на основе минимального остовного дерева и генетический алгоритм с настраиваемыми эвристиками.

3.1 Представление данных и оценка решений

Для работы алгоритмов полный взвешенный граф задаётся матрицей расстояний D , где элемент $d_{i,j}$ вычисляется как евклидово расстояние между точками i и j на плоскости.

Решение (хромосома) в генетическом алгоритме является по сути перестановкой $\pi = (\pi_0, \pi_1, \dots, \pi_{n-1})$ индексов вершин от 0 до $n - 1$. Суммарная длина такого маршрута $L(\pi)$ вычисляется с учётом замыкания цикла:

$$L(\pi) = \sum_{i=0}^{n-1} d_{\pi_i, \pi_{(i-1) \pmod n}}$$

Функция приспособленности $F(\pi)$, которую алгоритм стремится максимизировать, обратно пропорциональна длине маршрута:

$$F(\pi) = \frac{1}{L(\pi)}$$

3.2 Алгоритм 2-приближения

Я реализовал базовый алгоритм, с которым будет проводится сравнение - алгоритм 2-приближения, он работает следующим образом:

1. На основе матрицы расстояний строится полное остовное дерево.
2. С помощью алгоритма Краскала или Прима выделяется минимальное остовное дерево (в данной реализации используется функционал графовой библиотеки).
3. Выполняется обход остовного дерева в глубину. Порядок первого посещения вершин при обходе формирует итоговый гамильтонов цикл.

В силу неравенства треугольника, полученный маршрут гарантированно не превышает вес оптимального решения более чем в 2 раза.

3.3 Алгоритм локального поиска 2-Opt

Для сравнения с генетическим алгоритмом реализован классический метод локальной оптимизации 2-Opt [1]. Алгоритм начинается с некоторого

го начального решения (в данной работе используется результат MST-алгоритма) и итеративно улучшает его с помощью разворота подотрезков маршрута.

На каждой итерации перебираются все пары индексов $1 \leq i < j < n$. Если разворот подотрезка $[i, j]$ уменьшает общую длину маршрута, изменение применяется. Процесс повторяется до тех пор, пока ни один разворот не даёт улучшения или не будет достигнут лимит итераций.

Формально, для маршрута $\pi = (\pi_0, \dots, \pi_{n-1})$ операция 2-Opt с параметрами (i, j) создаёт новый маршрут:

$$\pi' = (\pi_0, \dots, \pi_{i-1}, \pi_j, \pi_{j-1}, \dots, \pi_i, \pi_{j+1}, \dots, \pi_{n-1})$$

Алгоритм гарантированно находит локальный оптимум относительно оператора 2-Opt, но не даёт теоретических гарантий приближения к глобальному оптимуму. На практике 2-Opt часто используется как пост-обработка для улучшения решений, полученных другими эвристиками.

3.4 Инициализация популяции

Начальная популяция генетического алгоритма формируется двумя способами:

- **Случайная инициализация:** хромосома генерируется путём случайного перемешивания индексов городов.
- **Смешанная инициализация:** для предотвращения быстрого вырождения популяции, но обеспечения быстрого старта, 10% особей создаются с помощью жадного алгоритма «ближайшего соседа» (начиная со случайной вершины, алгоритм всегда переходит в ближайшую непосещённую), а остальные 90% - случайным образом.

3.5 Операторы генетического алгоритма

Основной цикл генетического алгоритма состоит из отбора, скрещивания и мутации.

1. Отбор. Я использовал турнирный отбор с размером турнира $k = 3$. Из популяции случайно выбираются 3 особи, и для дальнейшего размножения отбирается та, чья функция приспособленности максимальна.

Дополнительно применяется *элитизм*: две лучшие особи текущего поколения гарантированно переходят в следующее поколение без изменений, это гарантирует, что мы не потеряем оптимум.

2. Скрещивание. Обычное одноточечное скрещивание неприменимо для задачи TSP, так как оно приводит к появлению дубликатов городов в маршруте, поэтому реализованы следующие виды скрещивания:

- **Ordered Crossover:** Выбирается случайный подотрезок в хромосоме первого родителя и копируется потомку без изменений. Оставшиеся пустые позиции заполняются генами второго родителя в порядке их следования, при условии, что они ещё не присутствуют в хромосоме потомка. Этот метод отлично сохраняет относительный порядок городов.
- **Partially Mapped Crossover:** Как и в Ordered Crossover, копируется случайный участок первого родителя. Затем строится взаимно однозначное отображение генов между скопированным участком первого и второго родителей. Оставшиеся позиции заполняются генами второго родителя, а при возникновении конфликтов используется построенное отображение для поиска легального города.

3. Мутация. С заданной вероятностью к потомкам применяется мутация для поддержания генетического разнообразия:

- **Swap-мутация:** случайно выбираются две позиции в хромосоме, и находящиеся в них города меняются местами.
- **Inversion-мутация:** случайно выбирается подотрезок хромосомы и его порядок инвертируется. В контексте метрической задачи коммивояжёра эта мутация по сути эквивалентна распутыванию перекрещивающихся рёбер.
- **Scramble-мутация:** элементы внутри случайно выбранного подотрезка перемешиваются случайным образом.

4 Эксперимент и анализ результатов

Для проверки гипотез алгоритм протестирован на сгенерированных наборах данных: равномерное распределение, кластерное распределение,

кольцевое расположение точек, а также на малом графе ($N=8$) для сравнения с точным решением. Результаты генетического алгоритма сравниваются с алгоритмом MST. Статистика (минимум, максимум, среднее время и длина пути) собирается на основе 40 независимых запусков для каждого набора.

На рисунке 1 показаны графики сходимости лучшего найденного маршрута по поколениям для всех трёх конфигураций на разных тестовых множествах. Чтобы линии не сливались, для каждого тест-сета построен отдельный подграфик, а ось ординат иногда приведена к логарифмическому масштабу в тех местах, где разброс значений велик.

4.1 Генерация тестовых данных

Для экспериментов использовались четыре типа наборов данных:

- **Малый граф ($N=8$):** 8 точек со случайными координатами в квадрате $[0,100]$, сгенерированные с фиксированным $\text{seed}=42$ для воспроизводимости. Для этого набора известно точное оптимальное решение, полученное полным перебором.
- **Равномерный ($N=50$):** 50 точек, равномерно распределённых в квадрате $[0,100]-[0,100]$.
- **Кластерный ($N=50$):** 50 точек, сгруппированных в 4 кластера. Центры кластеров выбираются случайно в квадрате $[0,100]$, точки внутри каждого кластера генерируются по нормальному распределению со стандартным отклонением 5 относительно центра.
- **Кольцевой ($N=50$):** 50 точек, равномерно распределённых по окружности радиуса 40 с центром в $(50,50)$. Этот тип графа имеет очевидную структуру оптимального маршрута.

Модели случайных графов

Для исследования поведения алгоритмов на графах различной структуры использованы следующие модели:

- **Erdős-Rényi $G(n, p)$:** случайный граф с вероятностью ребра p . Для получения метрической задачи вычисляются кратчайшие пути между всеми парами вершин (с помощью алгоритма Флойда-Уоршелла). Рассмотрены три режима:

- $p = 0.3$ - плотный граф, средняя степень $\Theta(n)$;
- $p = 1/n$ - разреженный граф, средняя степень постоянна;
- $p = \ln n/n$ - пороговая вероятность: при "меньших" значениях граф несвязен, при "больших" связан.

Все графы приводятся к связному виду добавлением случайных рёбер при необходимости.

Расстояния между вершинами вычисляются как евклидовы расстояния между соответствующими точками на плоскости, поэтому неравенство треугольника выполняется и решаемая задача действительно метрическая.

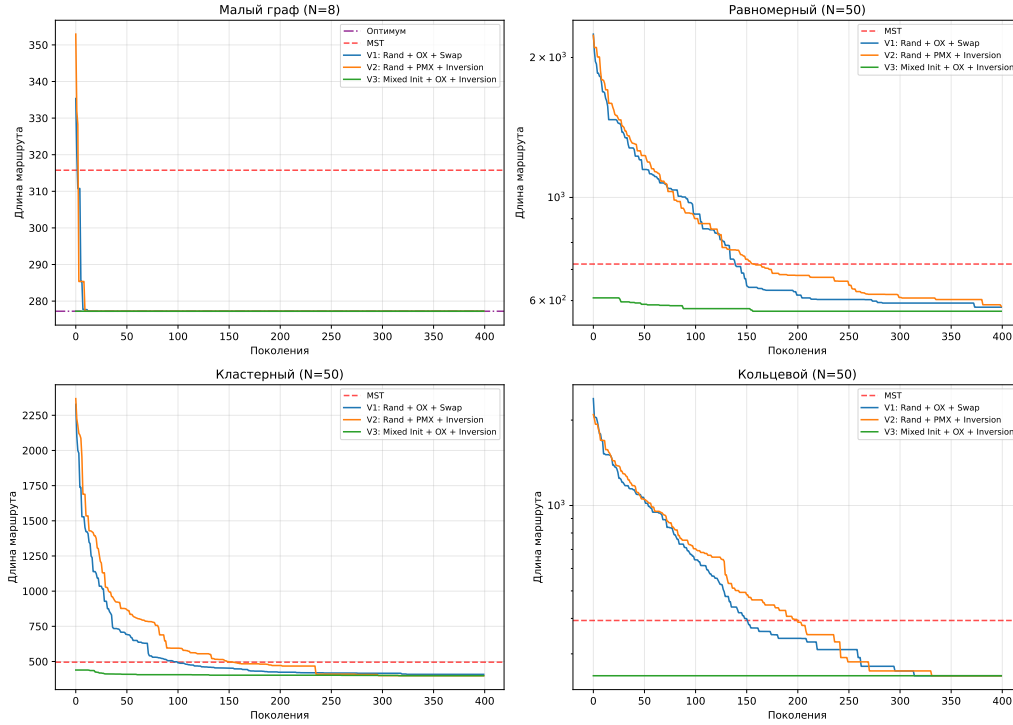


Рис. 1: Сходимость генетического алгоритма для разных конфигураций и наборов данных. Пунктиром обозначен результат MST-приближения, фиолетовой линией - оптимум там, где он известен.

Отметим, что для малого графа ($N=8$) все три конфигурации генетического алгоритма находят оптимальное решение, поэтому на соответ-

ствующем подграфике линии сходимости практически совпадают. Это подтверждает корректность реализации: на простых инстансах даже базовые эвристики способны найти глобальный оптимум.

Полный вывод программы с числовыми результатами приведён в приложении. Ниже находятся основные наблюдения.

Влияние операторов мутации: Как показали тесты, Inversion-мутация сходится быстрее и находит более качественные решения, чем Swap. Это происходит из-за того, что для метрической TSP разворот пути устраняет пересечения рёбер, сохраняя при этом длины соседних связей, а Swap разрушает локальную структуру маршрута.

На кластерных и кольцевых графах комбинация смешанной инициализации и Inversion-мутации (V3) практически всегда сходится к глобальному оптимуму, демонстрируя улучшение относительно MST на 18-36%. Случайная инициализация с Swap (V1) показывает значительно худшие результаты и большой разброс, а на кольцевом графе даже уступает MST.

Эксперимент по масштабируемости

Для оценки зависимости качества решений от размера графа проведено тестирование на графах с $n \in \{10, 20, 30, 40, 50, 100\}$ вершинами. Для каждого n и типа графа сравнивались три алгоритма:

- MST-приближение;
- Генетический алгоритм (с разнообразными конфигурациями);
- Локальный поиск 2-Opt, применяемый к MST.

Результаты усреднялись по типам графов, графики зависимости длины маршрута и времени выполнения от n приведены на рисунке 2.

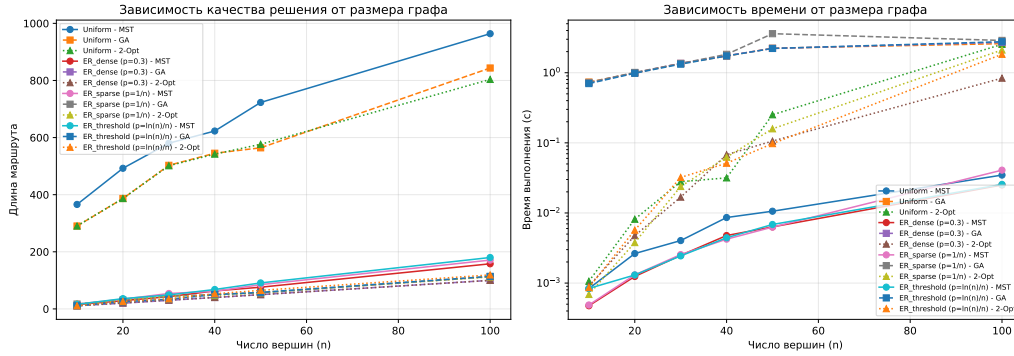


Рис. 2: Зависимость качества решения (слева) и времени выполнения (справа) от числа вершин n для различных типов графов. Линии с маркерами $\circ$, $\square$, $\triangle$ соответствуют алгоритмам MST, GA и 2-Opt соответственно.

Анализ результатов. Как видно из таблицы 1 и графика 2, оба эвристических алгоритма (GA и 2-Opt) стабильно превосходят базовое MST-приближение.

- **На равномерных графах** улучшение составляет 12-22% для обоих методов, причём на больших n 2-Opt иногда показывает чуть лучший результат.
- **На плотных случайных графах** ($p = 0.3$) оба метода дают улучшение 26-37%, причём результаты GA и 2-Opt практически совпадают. Получается, что в плотных графах локальная структура решения хорошо оптимизируется даже простыми методами.
- **На разреженных графах** ($p = 1/n$) при малых n улучшение незначительно, но растёт с увеличением размера: при $n = 100$ оба метода улучшают MST на 34%.
- **Временная сложность:** MST работает за миллисекунды даже при $n = 100$, GA требует 1-3 секунды, 2-Opt - доли секунды. Таким образом, 2-Opt является эффективным компромиссом между качеством и скоростью.

Таблица 1: Среднее улучшение относительно MST для разных типов графов и размеров

2*Тип графа	GA vs MST			2-Opt vs MST		
	$n = 10$	$n = 50$	$n = 100$	$n = 10$	$n = 50$	$n = 100$
Uniform	20.6%	22.0%	12.5%	20.6%	20.3%	16.6%
ER _{dense} ($p=0.3$)	26.7%	34.2%	36.7%	26.7%	34.2%	36.7%
ER _{sparse} ($p=1/n$)	0.0%	34.1%	33.9%	0.0%	32.9%	34.5%
ER_threshold	31.2%	36.3%	36.7%	31.2%	28.6%	33.9%

Сравнение с MST: Алгоритм MST работает за время $O(V^2)$ и всегда укладывается в миллисекунды. Генетический алгоритм требует больше времени (секунды), однако итоговая длина пути в ГА с Inversion-мутацией стабильно оказывается на 15-25% короче, чем результат 2-приближённого MST-алгоритма. На кластерных графах ГА иногда требует большего размера популяции для преодоления локальных оптимумов, что видно по медленной сходимости на начальных поколениях.

5 Заключение и выводы

В ходе работы был реализован генетический алгоритм для решения метрической задачи коммивояжёра. Мы получили, что правильный выбор эвристик, такие как Inversion мутация вместо Swap очень важен для метрических графов. Генетический алгоритм оказался ощутимо точнее базового 2-приближённого подхода, поэтому он эффективен в оптимизационных задачах на графах.

При дополнительном сравнении с алгоритмом локального поиска 2-Opt можно увидеть, что для многих типов графов (особенно плотных случайных) простой детерминированный метод пост-обработки даёт достаточно хороший результат, сопоставимый с генетическим алгоритмом, но за ощутимо меньшее время. Поэтому возможно окажется удачным гибридный подход: можно использовать генетический алгоритм для поиска глобально перспективных областей решения, а 2-Opt - для быстрой локальной доводки.

А Приложение

А.1 Исходный код программы (Gen.py)

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 import networkx as nx
4 from scipy.spatial.distance import pdist, squareform
5 import time
6 import random
7 from itertools import permutations
8
9 def generate_uniform_tsp(n_cities, seed=None):
10     """Создаёт n_cities точек со случайными координатами в
11     ↪ квадрате [0,100]."""
12     if seed: np.random.seed(seed)
13     return np.random.rand(n_cities, 2) * 100
14
15 def generate_clustered_tsp(n_cities, n_clusters=4, seed=None):
16     """Генерирует данные с несколькими плотными кластерами."""
17     if seed: np.random.seed(seed)
18     points = []
19     cities_per_cluster = n_cities // n_clusters
20     for _ in range(n_clusters):
21         center = np.random.rand(2) * 100
22         cluster_points = center +
23         ↪ np.random.randn(cities_per_cluster, 2) * 5
24         points.extend(cluster_points)
25     return np.array(points)
26
27 def generate_circular_tsp(n_cities):
28     """Равномерно расставляет города по окружности радиуса 40 с
29     ↪ центром (50,50)."""
30     angles = np.linspace(0, 2 * np.pi, n_cities, endpoint=False)
31     x = 50 + 40 * np.cos(angles)
32     y = 50 + 40 * np.sin(angles)
33     return np.column_stack((x, y))
34
35 def generate_erdos_renyi_tsp(n_cities, p, seed=None):
36     """
```

```

34     Генерирует метрический TSP-экземпляр на основе модели
        ↳ Эрдёша-Реньи  $G(n, p)$ .
35     Сначала строится случайный граф, затем вычисляются кратчайшие
        ↳ пути (алг. Флойд-Уоршелл)
36     для получения метрической матрицы расстояний.
37     """
38     if seed: np.random.seed(seed)
39     G = nx.erdos_renyi_graph(n_cities, p, seed=seed)
40
41     if not nx.is_connected(G):
42         # Добавляем случайные рёбра пока граф не станет связным
43         nodes = list(G.nodes())
44         while not nx.is_connected(G):
45             u, v = random.sample(nodes, 2)
46             if not G.has_edge(u, v):
47                 G.add_edge(u, v, weight=1.0)
48
49     # Вычисляем матрицу кратчайших путей
50     dist_matrix = np.ones((n_cities, n_cities)) * np.inf
51     np.fill_diagonal(dist_matrix, 0)
52     for u, v in G.edges():
53         dist_matrix[u, v] = dist_matrix[v, u] = 1.0
54
55     # Алгоритм Флойда-Уоршелла для метризации
56     for k in range(n_cities):
57         for i in range(n_cities):
58             for j in range(n_cities):
59                 if dist_matrix[i, j] > dist_matrix[i, k] +
                    ↳ dist_matrix[k, j]:
60                     dist_matrix[i, j] = dist_matrix[i, k] +
                    ↳ dist_matrix[k, j]
61
62     return dist_matrix
63
64 def generate_configuration_model_tsp(n_cities, degree_sequence,
    ↳ seed=None):
65     """
66     Генерирует метрический TSP на основе конфигурационной модели.
67     degree_sequence: список желаемых степеней вершин.
68     """

```

```

69     if seed: random.seed(seed)
70     G = nx.configuration_model(degree_sequence, seed=seed,
    ↪     create_using=nx.Graph())
71     G = nx.Graph(G)
72     G.remove_edges_from(nx.selfloop_edges(G))
73
74     # Если граф несвязный - добавляем рёбра
75     if not nx.is_connected(G):
76         nodes = list(G.nodes())
77         while not nx.is_connected(G):
78             u, v = random.sample(nodes, 2)
79             if not G.has_edge(u, v):
80                 G.add_edge(u, v)
81
82     # Метризация через кратчайшие пути
83     dist_matrix = nx.floyd_warshall_numpy(G)
84     return dist_matrix
85
86
87 def get_distance_matrix(points):
88     """Вычисляет матрицу евклидовых расстояний."""
89     return squareform(pdist(points, 'euclidean'))
90
91 def path_length(path, dist_matrix):
92     """Возвращает длину замкнутого маршрута."""
93     return sum(dist_matrix[path[i], path[i-1]] for i in
    ↪     range(len(path)))
94
95 def solve_tsp_brute_force(dist_matrix):
96     """Точный перебор всех перестановок (для N <= 10)."""
97     n = len(dist_matrix)
98     min_dist = float('inf')
99     best_path = None
100     for path in permutations(range(n)):
101         dist = path_length(path, dist_matrix)
102         if dist < min_dist:
103             min_dist = dist
104             best_path = path
105     return list(best_path), min_dist
106

```

```

107 def solve_tsp_mst(dist_matrix):
108     """2-приближение через минимальное остовное дерево."""
109     n = len(dist_matrix)
110     G = nx.from_numpy_array(dist_matrix)
111     mst = nx.minimum_spanning_tree(G)
112     path = list(nx.dfs_preorder_nodes(mst, source=0))
113     return path, path_length(path, dist_matrix)
114
115
116 def solve_tsp_2opt(dist_matrix, initial_path=None,
117     ↪ max_iterations=1000):
118     """
119     Улучшение маршрута с помощью локального поиска 2-Opt.
120     Если initial_path не задан, начинает с жадного решения.
121     """
122     n = len(dist_matrix)
123
124     # Начальное решение: жадный алгоритм или переданный путь
125     if initial_path is None:
126         start = 0
127         unvisited = set(range(n))
128         unvisited.remove(start)
129         path = [start]
130         current = start
131         while unvisited:
132             next_city = min(unvisited, key=lambda city:
133                 ↪ dist_matrix[current, city])
134             path.append(next_city)
135             unvisited.remove(next_city)
136             current = next_city
137     else:
138         path = list(initial_path)
139
140     best_path = path
141     best_dist = path_length(best_path, dist_matrix)
142
143     # Итеративное улучшение 2-Opt
144     improved = True
145     iteration = 0
146     while improved and iteration < max_iterations:

```



```

145         improved = False
146         iteration += 1
147         for i in range(1, n - 1):
148             for j in range(i + 1, n):
149                 # Пробуем развернуть подотрезок [i, j]
150                 new_path = best_path[:i] + best_path[i:j+1][::-1]
151                 ↪ + best_path[j+1:]
152                 new_dist = path_length(new_path, dist_matrix)
153                 if new_dist < best_dist - 1e-8:
154                     best_path = new_path
155                     best_dist = new_dist
156                     improved = True
157                     break
158             if improved:
159                 break
160         return best_path, best_dist
161
162
163 class TSPGeneticAlgorithm:
164     """Генетический алгоритм для метрической задачи
165     ↪ коммивояжёра."""
166     def __init__(self, dist_matrix, pop_size=100, generations=300,
167                 mutation_rate=0.2, init_type='random',
168                 crossover_type='ox', mutation_type='inversion'):
169         self.dist_matrix = dist_matrix
170         self.n_cities = len(dist_matrix)
171         self.pop_size = pop_size
172         self.generations = generations
173         self.mutation_rate = mutation_rate
174         self.init_type = init_type
175         self.crossover_type = crossover_type
176         self.mutation_type = mutation_type
177
178     def _create_individual(self, use_greedy=False):
179         """Создаёт особь: либо жадным алгоритмом, либо случайной
180         ↪ перестановкой."""
181         if use_greedy:
182             start = random.randint(0, self.n_cities - 1)
183             unvisited = set(range(self.n_cities))

```

```

182         unvisited.remove(start)
183         path = [start]
184         current = start
185         while unvisited:
186             next_city = min(unvisited, key=lambda city:
187                             ↪ self.dist_matrix[current, city])
187             path.append(next_city)
188             unvisited.remove(next_city)
189             current = next_city
190         return path
191     else:
192         ind = list(range(self.n_cities))
193         random.shuffle(ind)
194         return ind
195
196     def _init_population(self):
197         """Формирует начальную популяцию с учётом init_type."""
198         population = []
199         for i in range(self.pop_size):
200             use_greedy = (self.init_type == 'mixed' and i <
201                           ↪ self.pop_size * 0.1)
201             population.append(self._create_individual(use_greedy=
202               ↪ use_greedy))
202         return population
203
204     def _fitness(self, individual):
205         """Приспособленность = 1 / длина маршрута."""
206         return 1.0 / path_length(individual, self.dist_matrix)
207
208     def _tournament_selection(self, population, fitnesses, k=3):
209         """Турнирный отбор размера k."""
210         selected_indices = random.sample(range(self.pop_size), k)
211         best_idx = max(selected_indices, key=lambda idx:
212                       ↪ fitnesses[idx])
212         return population[best_idx]
213
214     def _ordered_crossover(self, p1, p2):
215         """Ordered crossover: сохраняет подотрезок первого
216         ↪ родителя, остальное заполняется вторым."""

```

```

216         start, end = sorted(random.sample(range(self.n_cities),
↪ 2))
217         child = [-1] * self.n_cities
218         child[start:end] = p1[start:end]
219         p2_idx, child_idx = end, end
220         while -1 in child:
221             if p2[p2_idx % self.n_cities] not in child:
222                 child[child_idx % self.n_cities] = p2[p2_idx %
↪ self.n_cities]
223                 child_idx += 1
224             p2_idx += 1
225         return child
226
227     def _pmx_crossover(self, p1, p2):
228         """Partially Mapped crossover: строит отображение между
↪ подотрезками."""
229         start, end = sorted(random.sample(range(self.n_cities),
↪ 2))
230         child = [-1] * self.n_cities
231         child[start:end] = p1[start:end]
232         mapping = {p1[i]: p2[i] for i in range(start, end)}
233
234         for i in range(self.n_cities):
235             if start <= i < end: continue
236             val = p2[i]
237             while val in mapping:
238                 val = mapping[val]
239             child[i] = val
240         return child
241
242     def _mutate(self, individual):
243         """Применяет мутацию с вероятностью mutation_rate."""
244         if random.random() < self.mutation_rate:
245             idx1, idx2 =
↪ sorted(random.sample(range(self.n_cities), 2))
246             if self.mutation_type == 'swap':
247                 individual[idx1], individual[idx2] =
↪ individual[idx2], individual[idx1]
248             elif self.mutation_type == 'inversion':

```

```

249         individual[idx1:idx2] =
250             ↪ reversed(individual[idx1:idx2])
251     elif self.mutation_type == 'scramble':
252         sub = individual[idx1:idx2]
253         random.shuffle(sub)
254         individual[idx1:idx2] = sub
255     return individual
256
257 def run(self):
258     """Запуск ГА, возвращает лучший маршрут, его длину и
259     ↪ историю улучшений."""
260     population = self._init_population()
261     best_path = None
262     best_dist = float('inf')
263     history_best = []
264
265     for _ in range(self.generations):
266         fitnesses = [self._fitness(ind) for ind in population]
267
268         current_best_idx = np.argmax(fitnesses)
269         current_best_dist =
270             ↪ path_length(population[current_best_idx],
271             ↪ self.dist_matrix)
272
273         if current_best_dist < best_dist:
274             best_dist = current_best_dist
275             best_path = list(population[current_best_idx])
276
277         history_best.append(best_dist)
278
279         sorted_indices = np.argsort(fitnesses)[::-1]
280         new_population = [population[sorted_indices[0]],
281             ↪ population[sorted_indices[1]]]
282
283         while len(new_population) < self.pop_size:
284             p1 = self._tournament_selection(population,
285                 ↪ fitnesses)
286             p2 = self._tournament_selection(population,
287                 ↪ fitnesses)

```

```

282         if self.crossover_type == 'pmx':
283             child = self._pmx_crossover(p1, p2)
284         else:
285             child = self._ordered_crossover(p1, p2)
286
287         child = self._mutate(child)
288         new_population.append(child)
289
290         population = new_population
291
292         return best_path, best_dist, history_best
293
294
295 def run_experiments():
296     """Главная функция: прогоняет эксперименты и строит графики
297     ↪ сходимости."""
298     np.random.seed(42)
299     random.seed(42)
300
301     configs = [
302         {'name': 'V1: Rand + OX + Swap', 'init': 'random',
303          ↪ 'cross': 'ox', 'mut': 'swap'},
304         {'name': 'V2: Rand + PMX + Inversion', 'init': 'random',
305          ↪ 'cross': 'pmx', 'mut': 'inversion'},
306         {'name': 'V3: Mixed Init + OX + Inversion', 'init':
307          ↪ 'mixed', 'cross': 'ox', 'mut': 'inversion'}
308     ]
309
310     testsets = {
311         "Малый граф (N=8)":
312         ↪ get_distance_matrix(generate_uniform_tsp(8)),
313         "Равномерный (N=50)":
314         ↪ get_distance_matrix(generate_uniform_tsp(50)),
315         "Кластерный (N=50)":
316         ↪ get_distance_matrix(generate_clustered_tsp(50, 4)),
317         "Кольцевой (N=50)":
318         ↪ get_distance_matrix(generate_circular_tsp(50))
319     }
320
321     runs_per_test = 40

```

```

314
315     # Построение сетки графиков
316     fig, axes = plt.subplots(2, 2, figsize=(14, 10))
317     axes = axes.flatten()
318
319     for idx, (ts_name, dist_mat) in enumerate(testsets.items()):
320         ax = axes[idx]
321         ax.set_title(ts_name)
322
323         print(f"\n{'='*60}\nТЕСТ-СЕТ: {ts_name}\n{'='*60}")
324
325         if len(dist_mat) <= 10:
326             _, opt_dist = solve_tsp_brute_force(dist_mat)
327             print(f"[Точный оптимум (Полный перебор)]:
328                 ↪ {opt_dist:.2f}")
329             ax.axhline(
330                 y=opt_dist,
331                 color='purple',
332                 linestyle='-.',
333                 linewidth=1.5,
334                 alpha=0.8,
335                 label='Оптимум'
336             )
337         else:
338             opt_dist = None
339
340         t0 = time.time()
341         _, mst_dist = solve_tsp_mst(dist_mat)
342         t_mst = time.time() - t0
343         print(f"[MST 2-приближение]: {mst_dist:.2f} (Время:
344             ↪ {t_mst:.4f} с)")
345         ax.axhline(y=mst_dist, color='red', linestyle='--',
346             ↪ alpha=0.7, label='MST')
347
348         for conf in configs:
349             dists, times = [], []
350             best_hist = None
351             best_overall = float('inf')
352
353             for _ in range(runs_per_test):

```

```

351         ga = TSPGeneticAlgorithm(dist_mat, pop_size=100,
352             ↪ generations=400,
353                                     init_type=conf['init'],
354                                     ↪ crossover_type=conf[
355                                     ↪ 'cross'],
356                                     mutation_type=conf['mut']
357                                     ↪ ])
358
359         t0 = time.time()
360         _, dist, hist = ga.run()
361         t_ga = time.time() - t0
362
363         dists.append(dist)
364         times.append(t_ga)
365         if dist < best_overall:
366             best_overall = dist
367             best_hist = hist
368
369         print(f"\n[{conf['name']}] Статистика
370             ↪ ({runs_per_test} запусков):")
371         print(f"    Мин: {np.min(dists):.2f} | Макс:
372             ↪ {np.max(dists):.2f} | Среднее:
373             ↪ {np.mean(dists):.2f}")
374         print(f"    Медиана: {np.median(dists):.2f} | Откл(Std):
375             ↪ {np.std(dists):.2f}")
376         print(f"    Ср. время: {np.mean(times):.2f} с")
377         print(f"    Улучшение от MST: {(mst_dist -
378             ↪ np.mean(dists)) / mst_dist * 100:.2f}%")
379
380         ax.plot(best_hist, label=conf['name'])
381
382         ax.set_xlabel('Поколения')
383         ax.set_ylabel('Длина маршрута')
384         ax.grid(True, alpha=0.3)
385         ax.legend(fontsize=8)
386
387         # Для графиков с большим разбросом используем
388         ↪ логарифмическую шкалу
389         if ts_name in ["Равномерный (N=50)", "Кольцевой (N=50)"]:
390             ax.set_yscale('log')

```

```

381 plt.tight_layout()
382 plt.savefig('convergence_plots.pdf', dpi=300)
383 plt.show()
384
385 # Тестирование масштабируемости и новых типов графов
386 print(f"\n{'='*60}\nМАСШТАБИРУЕМОСТЬ И НОВЫЕ ТИПЫ
    ↳ ГРАФОВ\n{'='*60}")
387
388 sizes = [10, 20, 30, 40, 50, 100]
389 graph_types = {
390     'Uniform': lambda n:
391         ↳ get_distance_matrix(generate_uniform_tsp(n, seed=42)),
392     'ER_dense (p=0.3)': lambda n: generate_erdos_renyi_tsp(n,
393         ↳ p=0.3, seed=42),
394     'ER_sparse (p=1/n)': lambda n: generate_erdos_renyi_tsp(n,
395         ↳ p=1.0/n, seed=42),
396     'ER_threshold (p=ln(n)/n)': lambda n:
397         ↳ generate_erdos_renyi_tsp(n, p=np.log(n)/n, seed=42),
398 }
399
400 results = {gt: {'n': [], 'mst_dist': [], 'ga_dist': [],
401     ↳ '2opt_dist': [],
402         'mst_time': [], 'ga_time': [], '2opt_time':
403         ↳ []}
404     for gt in graph_types}
405
406 for gt_name, gen_func in graph_types.items():
407     print(f"\n--- Тип графа: {gt_name} ---")
408     for n in sizes:
409         print(f"  n={n}...", end=' ', flush=True)
410         dist_mat = gen_func(n)
411
412         # MST
413         t0 = time.time()
414         _, mst_dist = solve_tsp_mst(dist_mat)
415         mst_time = time.time() - t0
416
417         # GA
418         pop_size = 100 if n <= 50 else 50
419         generations = 400 if n <= 50 else 200

```



```

414     t0 = time.time()
415     ga = TSPGeneticAlgorithm(dist_mat, pop_size=pop_size,
416                             ↪ generations=generations,
417                             init_type='mixed',
418                             ↪ crossover_type='ox', mut_
419                             ↪ ation_type='inversion')
420
421     _, ga_dist, _ = ga.run()
422     ga_time = time.time() - t0
423
424     # 2-Opt
425     t0 = time.time()
426     mst_path, _ = solve_tsp_mst(dist_mat)
427     _, opt_dist = solve_tsp_2opt(dist_mat,
428     ↪ initial_path=mst_path)
429     opt_time = time.time() - t0
430
431     results[gt_name]['n'].append(n)
432     results[gt_name]['mst_dist'].append(mst_dist)
433     results[gt_name]['ga_dist'].append(ga_dist)
434     results[gt_name]['2opt_dist'].append(opt_dist)
435     results[gt_name]['mst_time'].append(mst_time)
436     results[gt_name]['ga_time'].append(ga_time)
437     results[gt_name]['2opt_time'].append(opt_time)
438
439     print(f"MST={mst_dist:.1f}, GA={ga_dist:.1f},
440     ↪ 2-Opt={opt_dist:.1f}")
441
442     # Построение графика масштабируемости
443     fig2, axes2 = plt.subplots(1, 2, figsize=(14, 5))
444
445     # График 1: Длина маршрута от n
446     ax1 = axes2[0]
447     for gt_name in graph_types:
448         ax1.plot(results[gt_name]['n'],
449         ↪ results[gt_name]['mst_dist'],
450         ↪ marker='o', label=f'{gt_name} - MST',
451         ↪ linestyle='-')
452     ax1.plot(results[gt_name]['n'],
453     ↪ results[gt_name]['ga_dist'],

```

```

445         marker='s', label=f'{gt_name} - GA',
           ↪ linestyle='--')
446     ax1.plot(results[gt_name]['n'],
           ↪ results[gt_name]['2opt_dist'],
447         marker='^', label=f'{gt_name} - 2-Opt',
           ↪ linestyle=':')
448     ax1.set_xlabel('Число вершин (n)')
449     ax1.set_ylabel('Длина маршрута')
450     ax1.set_title('Зависимость качества решения от размера графа')
451     ax1.grid(True, alpha=0.3)
452     ax1.legend(fontsize=7)
453
454     # График 2: Время выполнения от n с логарифмической шкалой
455     ax2 = axes2[1]
456     for gt_name in graph_types:
457         ax2.plot(results[gt_name]['n'],
           ↪ results[gt_name]['mst_time'],
458             marker='o', label=f'{gt_name} - MST',
           ↪ linestyle='-')
459         ax2.plot(results[gt_name]['n'],
           ↪ results[gt_name]['ga_time'],
460             marker='s', label=f'{gt_name} - GA',
           ↪ linestyle='--')
461         ax2.plot(results[gt_name]['n'],
           ↪ results[gt_name]['2opt_time'],
462             marker='^', label=f'{gt_name} - 2-Opt',
           ↪ linestyle=':')
463     ax2.set_xlabel('Число вершин (n)')
464     ax2.set_ylabel('Время выполнения (с)')
465     ax2.set_title('Зависимость времени от размера графа')
466     ax2.set_yscale('log')
467     ax2.grid(True, alpha=0.3)
468     ax2.legend(fontsize=7)
469
470     plt.tight_layout()
471     plt.savefig('scalability_plot.pdf', dpi=300)
472     plt.show()
473
474     # Вывод таблицы результатов

```

```

475     print(f"\n{' '*60}\nТАБЛИЦА РЕЗУЛЬТАТОВ (среднее улучшение
      ↳ относительно MST)\n{' '*60}")
476     for gt_name in graph_types:
477         print(f"\n{gt_name}:")
478         print(f"    {'n':>4} | {'MST':>8} | {'GA':>8} |
      ↳ {'2-Opt':>8} | {'GA vs MST':>10} | {'2-Opt vs
      ↳ MST':>12}")
479         print(f"    {'-'*4}-+-'-'*8}-+-'-'*8}-+-'-'*8}-+-'-'*10
      ↳ }-+-'-'*12}")
480         for i, n in enumerate(sizes):
481             mst = results[gt_name]['mst_dist'][i]
482             ga = results[gt_name]['ga_dist'][i]
483             opt = results[gt_name]['2opt_dist'][i]
484             ga_imp = (mst - ga) / mst * 100
485             opt_imp = (mst - opt) / mst * 100
486             print(f"    {n:>4} | {mst:>8.1f} | {ga:>8.1f} |
      ↳ {opt:>8.1f} | {ga_imp:>9.1f}% |
      ↳ {opt_imp:>11.1f}%")
487
488
489     if __name__ == "__main__":
490         run_experiments()

```

А.2 Результат выполнения

```

=====
ТЕСТ-СЕТ: Малый граф (N=8)
=====

[Точный оптимум (Полный перебор)]: 277.23
[MST 2-приближение]: 315.76 (Время: 0.0005 с)

[V1: Rand + OX + Swap] Статистика (40 запусков):
  Мин: 277.23 | Макс: 277.23 | Среднее: 277.23
  Медиана: 277.23 | Откл(Std): 0.00
  Ср. время: 0.71 с
  Улучшение от MST: 12.20%

[V2: Rand + PMX + Inversion] Статистика (40 запусков):

```

Мин: 277.23 | Макс: 277.23 | Среднее: 277.23
Медиана: 277.23 | Откл(Std): 0.00
Ср. время: 0.71 с
Улучшение от MST: 12.20%

[V3: Mixed Init + OX + Inversion] Статистика (40 запусков):
Мин: 277.23 | Макс: 277.23 | Среднее: 277.23
Медиана: 277.23 | Откл(Std): 0.00
Ср. время: 0.73 с
Улучшение от MST: 12.20%

=====

ТЕСТ-СЕТ: Равномерный (N=50)

=====

[MST 2-приближение]: 718.97 (Время: 0.1633 с)

[V1: Rand + OX + Swap] Статистика (40 запусков):
Мин: 580.65 | Макс: 749.06 | Среднее: 680.49
Медиана: 669.02 | Откл(Std): 40.69
Ср. время: 2.54 с
Улучшение от MST: 5.35%

[V2: Rand + PMX + Inversion] Статистика (40 запусков):
Мин: 583.12 | Макс: 664.79 | Среднее: 615.91
Медиана: 616.85 | Откл(Std): 18.10
Ср. время: 1.46 с
Улучшение от MST: 14.33%

[V3: Mixed Init + OX + Inversion] Статистика (40 запусков):
Мин: 569.13 | Макс: 604.00 | Среднее: 585.82
Медиана: 584.87 | Откл(Std): 9.60
Ср. время: 2.48 с
Улучшение от MST: 18.52%

=====

ТЕСТ-СЕТ: Кластерный (N=50)

=====

[MST 2-приближение]: 495.16 (Время: 0.0074 с)

[V1: Rand + OX + Swap] Статистика (40 запусков):

Мин: 408.57 | Макс: 649.89 | Среднее: 481.84

Медиана: 474.54 | Откл(Std): 45.01

Ср. время: 2.47 с

Улучшение от MST: 2.69%

[V2: Rand + PMX + Inversion] Статистика (40 запусков):

Мин: 398.70 | Макс: 422.55 | Среднее: 408.55

Медиана: 407.97 | Откл(Std): 6.16

Ср. время: 1.39 с

Улучшение от MST: 17.49%

[V3: Mixed Init + OX + Inversion] Статистика (40 запусков):

Мин: 399.09 | Макс: 412.94 | Среднее: 402.85

Медиана: 401.58 | Откл(Std): 3.58

Ср. время: 2.40 с

Улучшение от MST: 18.64%

=====

ТЕСТ-СЕТ: Кольцевой (N=50)

=====

[MST 2-приближение]: 393.28 (Время: 0.0115 с)

[V1: Rand + OX + Swap] Статистика (40 запусков):

Мин: 251.16 | Макс: 566.18 | Среднее: 420.82

Медиана: 429.15 | Откл(Std): 71.34

Ср. время: 2.54 с

Улучшение от MST: -7.00%

[V2: Rand + PMX + Inversion] Статистика (40 запусков):

Мин: 251.16 | Макс: 391.25 | Среднее: 271.16

Медиана: 261.17 | Откл(Std): 31.49

Ср. время: 1.47 с

Улучшение от MST: 31.05%

[V3: Mixed Init + OX + Inversion] Статистика (40 запусков):

Мин: 251.16 | Макс: 251.16 | Среднее: 251.16

Медиана: 251.16 | Откл(Std): 0.00
Ср. время: 2.53 с
Улучшение от MST: 36.14%

=====

МАСШТАБИРУЕМОСТЬ И НОВЫЕ ТИПЫ ГРАФОВ

=====

--- Тип графа: Uniform ---

n=10...	MST=365.9,	GA=290.3,	2-Opt=290.3
n=20...	MST=492.5,	GA=386.6,	2-Opt=386.6
n=30...	MST=580.3,	GA=502.9,	2-Opt=501.3
n=40...	MST=623.0,	GA=545.2,	2-Opt=542.1
n=50...	MST=723.0,	GA=564.1,	2-Opt=576.5
n=100...	MST=963.8,	GA=843.6,	2-Opt=803.7

--- Тип графа: ER_dense (p=0.3) ---

n=10...	MST=15.0,	GA=11.0,	2-Opt=11.0
n=20...	MST=29.0,	GA=20.0,	2-Opt=21.0
n=30...	MST=43.0,	GA=30.0,	2-Opt=31.0
n=40...	MST=63.0,	GA=40.0,	2-Opt=40.0
n=50...	MST=76.0,	GA=50.0,	2-Opt=50.0
n=100...	MST=158.0,	GA=100.0,	2-Opt=100.0

--- Тип графа: ER_sparse (p=1/n) ---

n=10...	MST=17.0,	GA=17.0,	2-Opt=17.0
n=20...	MST=34.0,	GA=31.0,	2-Opt=31.0
n=30...	MST=54.0,	GA=44.0,	2-Opt=44.0
n=40...	MST=62.0,	GA=51.0,	2-Opt=47.0
n=50...	MST=85.0,	GA=56.0,	2-Opt=57.0
n=100...	MST=171.0,	GA=113.0,	2-Opt=112.0

--- Тип графа: ER_threshold (p=ln(n)/n) ---

n=10...	MST=16.0,	GA=11.0,	2-Opt=11.0
n=20...	MST=36.0,	GA=25.0,	2-Opt=26.0
n=30...	MST=49.0,	GA=34.0,	2-Opt=35.0
n=40...	MST=68.0,	GA=50.0,	2-Opt=53.0

n=50... MST=91.0, GA=58.0, 2-Opt=65.0
n=100... MST=180.0, GA=114.0, 2-Opt=119.0

ТАБЛИЦА РЕЗУЛЬТАТОВ (среднее улучшение относительно MST)

Uniform:

n	MST	GA	2-Opt	GA vs MST	2-Opt vs MST
10	365.9	290.3	290.3	20.6%	20.6%
20	492.5	386.6	386.6	21.5%	21.5%
30	580.3	502.9	501.3	13.3%	13.6%
40	623.0	545.2	542.1	12.5%	13.0%
50	723.0	564.1	576.5	22.0%	20.3%
100	963.8	843.6	803.7	12.5%	16.6%

ER_dense (p=0.3):

n	MST	GA	2-Opt	GA vs MST	2-Opt vs MST
10	15.0	11.0	11.0	26.7%	26.7%
20	29.0	20.0	21.0	31.0%	27.6%
30	43.0	30.0	31.0	30.2%	27.9%
40	63.0	40.0	40.0	36.5%	36.5%
50	76.0	50.0	50.0	34.2%	34.2%
100	158.0	100.0	100.0	36.7%	36.7%

ER_sparse (p=1/n):

n	MST	GA	2-Opt	GA vs MST	2-Opt vs MST
10	17.0	17.0	17.0	0.0%	0.0%
20	34.0	31.0	31.0	8.8%	8.8%
30	54.0	44.0	44.0	18.5%	18.5%
40	62.0	51.0	47.0	17.7%	24.2%
50	85.0	56.0	57.0	34.1%	32.9%
100	171.0	113.0	112.0	33.9%	34.5%

ER_threshold ($p=\ln(n)/n$):

n	MST	GA	2-Opt	GA vs MST	2-Opt vs MST
10	16.0	11.0	11.0	31.2%	31.2%
20	36.0	25.0	26.0	30.6%	27.8%
30	49.0	34.0	35.0	30.6%	28.6%
40	68.0	50.0	53.0	26.5%	22.1%
50	91.0	58.0	65.0	36.3%	28.6%
100	180.0	114.0	119.0	36.7%	33.9%

Список литературы

- [1] Кормен Т., Лейзерсон Ч., Ривест Р., Штайн К. Алгоритмы: построение и анализ. — М.: Вильямс, 2013.
- [2] Рутковская Д., Пилиньский М., Рутковский Л. Нейронные сети, генетические алгоритмы и нечеткие системы. — М.: Горячая линия - Телеком, 2004.